

Week #1: Welcome to Colab!

[Open this notebook in Colab](#)

Note: The first few sections of this week's notebook come from Google's [official introductory guide](#). With each passing week, the contents of these notebooks will get more and more specific to the aims of the internship.

What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

86400

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

604800

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Week 1 Practice: Python Basics

In this section, you will complete a few beginner exercises designed to build confidence with core Python skills.

For each exercise:

1. Write your code in the provided starter cell.
2. Click the **Check Exercise** button.
3. Read the feedback and revise if needed.

Most programmers need multiple tries when learning a new concept. Iteration is normal and expected.

> Setup checker utilities (run once)

[Show code](#)

Checker setup complete. You can now use the buttons below.

Lesson A: Variables and Arithmetic

A **variable** stores a value with a name:

```
seconds_in_minute = 60
```

You can combine variables with arithmetic:

```
seconds_in_hour = 60 * 60
seconds_in_two_hours = 2 * seconds_in_hour
```

Run the example cell below before attempting Exercise 1.

```
# Example: variable assignment + arithmetic

example_seconds_in_hour = 60 * 60
example_seconds_in_two_hours = 2 * example_seconds_in_hour

print(example_seconds_in_hour)      # 3600
print(example_seconds_in_two_hours) # 7200
```

3600
7200

Exercise A: Variables and Arithmetic

Create a variable named `seconds_in_hour` and set it equal to `60 * 60`.

Then create `seconds_in_two_hours` and set it to `2 * seconds_in_hour`.

Expected value for `seconds_in_two_hours`: **7200**.

```
# Write your solution here for Exercise 1
seconds_in_hour = 60 * 60
seconds_in_two_hours = 2 * seconds_in_hour
print (seconds_in_two_hours)
```

7200

> Check Exercise A

Show code

Check Exercise

Great work. Exercise 1 is correct.

Lesson B: Lists and `for` Loops

A **list** stores a sequence of values:

```
numbers = [1, 2, 3]
```

A **for loop** lets you go through each value in a list:

```
for n in numbers:
    print(n)
```

To build a total, create an accumulator variable and update it each loop iteration.

```
# Example: summing a list with a for loop

example_numbers = [4, 5, 6]
```

```
example_total = 0

for n in example_numbers:
    example_total = example_total + n

print(example_total) # 15
```

15

Exercise B: Lists and Loops

Create a list named `numbers` with values `[1, 2, 3, 4, 5]`.

Use a `for` loop to compute the sum of all values into a variable named `total`.

Expected value for `total`: 15.

```
# Write your solution here for Exercise 2
numbers = [1, 2, 3, 4, 5]
total = 0
for n in numbers:
    total = total + n
print (total)
```

15

Check Exercise B

[Show code](#)

Check Exercise

Nice job. Exercise 2 is correct.

Lesson C: Conditionals

A **conditional** lets your code make decisions based on a rule.

```
if x > 10:
    print("big")
else:
    print("small")
```

You can also chain multiple conditions using `elif`:

```
if score < 60:
    grade = "F"
elif score < 70:
    grade = "D"
else:
    grade = "C or above"
```

```
# Example: conditionals

temperature = 82

if temperature < 60:
    weather_label = "cold"
elif temperature <= 80:
    weather_label = "warm"
else:
    weather_label = "hot"

print(weather_label) # hot
```

hot

Exercise C: Conditionals

Set `temperature = 75`.

Then use `if` / `elif` / `else` to set a variable named `weather_label`:

- `"cold"` if temperature is less than 60
- `"warm"` if temperature is between 60 and 80 (inclusive)
- `"hot"` if temperature is greater than 80

```
# Write your solution here for Exercise C
temperature = 75
weather_label = None

if temperature < 60:
    weather_label = ("cold")
elif temperature <= 80:
    weather_label = ("warm")
else:
    weather_label = ("hot")

print(weather_label)

warm
```

> Check Exercise C

[Show code](#)

Check Exercise

Excellent work. Exercise C is correct.

Lesson D: Functions

A **function** is a reusable block of code. You define it once and call it many times.

```
def greet(name):
    return "Hello, " + name
```

Functions help you organize logic and avoid repeating the same code.

```
# Example: function definition and function call

def square(n):
    return n * n

print(square(3)) # 9
print(square(5)) # 25
```

Exercise D: Functions

Write a function named `is_even(n)` that:

- returns `True` when `n` is even
- returns `False` when `n` is odd

Use the modulo operator `%`.

```
# Write your solution here for Exercise D
def is_even(n):
    # Replace this with your implementation
    if n % 2 == 0:
        return True
    else:
        return False
```

> Check Exercise D

[Show code](#)

Check Exercise

Excellent work. Exercise D is correct.

Nice progress!

If all four checks pass, you are in a strong position for Week 2.

Optional stretch challenge:

- Create a `3 x 3` grid (a list of lists)
- Write a function that prints one row per line

If you get stuck, write down what you tried and where it broke. That is real engineering practice.

Wrapping Up

Before our next meeting:

1. Export this notebook as a PDF:
 - In Colab, go to **File > Print**
 - Save the print output as a PDF file
2. Email the PDF to your mentor:
 - michael.d.moffitt@gmail.com
3. In the same email, include:
 - Any questions you have
 - Any comments about what felt easy or difficult
 - Anything you want to practice more next week

✓ Stretch Challenge

Stretch challenge drafting

```
# Drafting to try and figure out solution.
# Final solution in next code box. *Found below

row1 = [1, 2, 3]
row2 = [4, 5, 6]
row3 = [7, 8, 9]

print (row1)
print (row2)
print (row3)

grid = row1 + row2 + row3
print (grid)

values = [1, 2, 3, 4, 5, 6, 7, 8, 9]
starting_num = 0

for n in values:

    fic_val = starting_num + n

    if fic_val <= 3:
        print (n)
    elif fic_val <= 6:
```

```

    print (n)
else:
    print (n)

```

```

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
1
2
3
4
5
6
7
8
9

```

```
#final solution.
```

```
values = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Row_1 = values[0:3]
```

```
Row_2 = values[3:6]
```

```
Row_3 = values[6:9]
```

```
grid = [Row_1, Row_2, Row_3]
```

```
for row in grid:
    print(row)
```

```

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

```

Challenges with stretch challenge:

- didn't realise that you could set ranges of one variable for another. Ex. Values[0:3] for Row_1
- Also not understanding that "n" and "row" are placeholders for values within the variable you have chosen with "in **grid**"